



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms

Dr. Iyappan Perumal

Assistant Professor Senior Grade 2

School of Computer Science & Engineering

VIT,Vellore.



Course Objective

1. To provide mathematical foundations for analysing the complexity of the algorithms
2. To impart the knowledge on various design strategies that can help in solving the real world problems effectively
3. To synthesize efficient algorithms in various engineering design situations



BCSE204L- Design and Analysis of Algorithms- Course outcome

1. Apply the mathematical tools to analyse and derive the running time of the algorithms
2. Demonstrate the major algorithm design paradigms.
3. Explain major graph algorithms, string matching and geometric algorithms along with their analysis.
4. Articulating Randomized Algorithms.
5. Explain the hardness of real-world problems with respect to algorithmic efficiency and learning to cope with it.



BCSE204L- Design and Analysis of Algorithms- Syllabus

Module:1	Design Paradigms: Greedy, Divide and Conquer Techniques	6 hours
Overview and Importance of Algorithms - Stages of algorithm development: Describing the problem, Identifying a suitable technique, Design of an algorithm, Derive Time Complexity, Proof of Correctness of the algorithm, Illustration of Design Stages - Greedy techniques: Fractional Knapsack Problem, and Huffman coding - Divide and Conquer: Maximum Subarray, Karatsuba faster integer multiplication algorithm.		
Module:2	Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques	10 hours
Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, Longest Common Subsequence, 0-1 Knapsack, TSP- Backtracking: N-Queens problem, Subset Sum, Graph Coloring- Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem		
Module:3	String Matching Algorithms	5 hours
Naïve String-matching Algorithms, KMP algorithm, Rabin-Karp Algorithm, Suffix Trees.		
Module:4	Graph Algorithms	6 hours
All pair shortest path: Bellman Ford Algorithm, Floyd-Warshall Algorithm - Network Flows: Flow Networks, Maximum Flows: Ford-Fulkerson, Edmond-Karp, Push Re-label Algorithm – Application of Max Flow to maximum matching problem		
Module:5	Geometric Algorithms	4 hours
Line Segments: Properties, Intersection, sweeping lines - Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm.		
Module:6	Randomized algorithms	5 hours
Randomized quick sort - The hiring problem - Finding the global Minimum Cut.		
Module:7	Classes of Complexity and Approximation Algorithms	7 hours
The Class P - The Class NP - Reducibility and NP-completeness – SAT (Problem Definition and statement), 3SAT, Independent Set, Clique, Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman		
Module:8	Contemporary Issues	2 hours

BCSE204L- Design and Analysis of Algorithms- References

Text Books:

1. Thomas H. Cormen, C.E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Reference Books:

1. Jon Kleinberg, Éva Tardos, Algorithm Design, Pearson education, 2014.
2. Rajeev Motwani, Prabhakar Raghavan; Randomized Algorithms, Cambridge University Press, 1995 (Online Print – 2013)
3. Ravindra K. Ahuja, Thomas L. Magnanti and James B. Orlin, “ Network Flows: Theory, Algorithms and Applications”, Pearson Education, 2014.



Module I: Design Paradigms: Greedy, Divide and Conquer Techniques(6 Hours)

- **Overview and Importance of Algorithms**
 - **Stages of algorithm development:**
 - Describing the problem
 - Identifying a suitable technique
 - Design of an algorithm
 - **Derive Time Complexity,**
 - Proof of Correctness of the algorithm
- **Illustration of Design Stages**
 - **Greedy techniques:**
 - Fractional Knapsack Problem
 - Huffman coding
 - **Divide and Conquer:**
 - Maximum Subarray
 - Karatsuba faster integer multiplication algorithm.



Time Complexity

- The amount of time taken by an algorithm to run to completion
- It is measured by number of primitive operations required to produce output.



Space complexity

- The amount of space required by an algorithm to run to completion.
- Mostly, time and space complexity is computed as a function of input size



Frequency Count/Step Count Method

- Number of times a statement is executed.
- RULES:
 - For comments, declarations, STEP COUNT=0
 - For return, assignment statements, STEP COUNT=1
 - Ignore lower order exponents when higher order exponents are present.
 - Ignore constant multipliers



Example: Sum of N Numbers(Iterative)

Algorithm	Step Count
Alg sum(a[],n)	--
{	--
sum=0;	1
for(i=1 to n) do	N+1
sum=sum+a[i];	N
return sum;	1
}	
TOTAL	2N+3
	O(N)



Example: Sum of N Numbers(Recursive)

Algorithm	N=0	N>0
Alg Rsum(a,n)	--	--
{	--	--
if(n<=0) then		
return 0;		--
else return Rsum(a,n-1)+a[n]	--	1+x
}	--	
TOTAL	2	2+x
	X=tRsum(n-1)	



Recurrences equation

- $$tRsum(n) = \begin{cases} 2 & \text{if } n=0 \\ 2+tRsum(n-1) & \text{if } n>0 \end{cases}$$

$$= 2+tRsum(n-1)$$

$$= 2+2+tRsum(n-2)$$

$$= 2+2+2+tRsum(n-3)$$

$$= n(2)+tRsum(0)$$

$$= 2n+2 = O(n)$$



Example: Sum of N Numbers(Space Complexity)

Algorithm

Alg sum(a[],n)

A[]– n words

{

n=one word

sum=0;

Sum=one word

for(i=1 to n) do

i= one word

sum=sum+a[i];

return sum;

}

TOTAL

N+3

O(n)



Recurrences and Running Time

- An equation that describes a function in terms of its value on smaller inputs.

Example: $T(n) = T(n-1) + n$

- Recurrences arise when an algorithm contains recursive calls to itself.
- What is the actual running time of the algorithm?
 - Need to solve the recurrence



Example Recurrences

- $T(n) = T(n-1) + n$

$\Theta(n^2)$

Recursive algorithm that loops through the input to eliminate one item

- $T(n) = T(n/2) + c$

$\Theta(\log n)$

Recursive algorithm that halves the input in one step

- $T(n) = T(n/2) + n$

$\Theta(n)$

Recursive algorithm that halves the input but must examine every item in the input

- $T(n) = 2T(n/2) + 1$

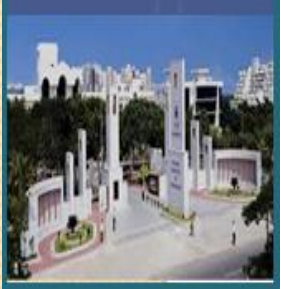
$\Theta(n)$

Recursive algorithm that splits the input into 2 halves and does a constant amount of other work



Orders of growth

$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < \dots < 2^n < 3^n < n^n$$





Example: Binary Search

Algorithm BINARY-SEARCH (A, low, high, x)

{

if (low > high) ----- Constant time: C_1

return FALSE

mid=(low+high)/2 ----- Constant time: C_2

if $x = A[\text{mid}]$ ----- Constant time: C_3

return TRUE

if ($x < A[\text{mid}]$)

BINARY-SEARCH (A, low, mid-1, x) --Same problem of size $n/2$

if ($x > A[\text{mid}]$)

BINARY-SEARCH (A, mid+1, high, x) --Same problem of size $n/2$

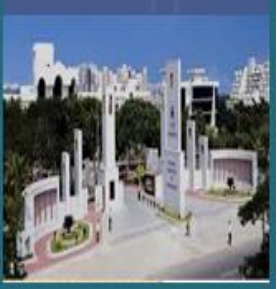
}

$$T(n) = C + T(n/2)$$

$T(n)$ – running time for an array of size n

Iteration Method

- Convert the recurrence into a summation and try to bound it using known series.
 - Iterate the recurrence until the initial condition is reached.
 - Use back-substitution to express the recurrence in terms of n and the initial (boundary) condition.



Iteration Method

$$\Rightarrow T(n) = C + T(n/2)$$

$$\Rightarrow T(n) = C + C + T(n/4) \quad \text{Where } T(n/2) = c + T(n/4)$$

$$\Rightarrow T(n) = C + C + C + T(n/8) \quad \text{where } T(n/4) = c + T(n/8)$$

.....

$$\Rightarrow T(n) = c + c + \dots + c + T(n/2^k)$$

k times

$$= K.C + T(1)$$

$$= C.\log n + 1$$

$$= \Theta(\log n)$$

Note: Where
 $n/2^k = 1$: Place not
divided further
so, $n = 2^k$



Iteration Method - Example

$$T(n) = n + 2T(n/2)$$

$$\begin{aligned} T(n) &= n + 2T(n/2) \\ &= n + 2(n/2 + 2T(n/4)) \text{ where } T(n/2) = n/2 + 2T(n/4) \\ &= n + n + 4T(n/4) \\ &= n + n + 4(n/4 + 2T(n/8)) \\ &= n + n + n + 8T(n/8) \end{aligned}$$

Pattern is repeated for i times

$$\begin{aligned} \dots &= i(n) + 2^i T(n/2^i) \\ &= i(n) + 2^i T(1) \\ &= n \log n + n T(1) \\ &= \Theta(n \log n) \end{aligned}$$



Master Method

- Master Theorem is the easiest way of obtaining runtime of recursive algorithms.
- General formula for the Master Theorem is:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

Number of times done

Division halves

Any other instructions/operation time taken

Where $a \geq 1$, $b \geq 1$ and $f(n) = \Theta(n^k \log^p n)$, $k \geq 1$

- Once, we have a , b and $f(n)$ we can determine the runtime of the work done by the recursion.

- It is given by

$$O(n^{\log_b a})$$



3 CASES

- CASE 1: $\text{Log}_b a > k$ $T(n) = \Theta(n^{\log_b a})$
- CASE 2: $\text{Log}_b a = k$
 - If $p > -1$, $T(n) = \Theta(n^k \log^{p+1} n)$
 - If $p = -1$, $T(n) = \Theta(n^k \log(\log n))$
 - If $p < -1$, $T(n) = \Theta(n^k)$
- CASE 3: $\text{Log}_b a < k$
 - If $p \geq 0$, $T(n) = \Theta(n^k \log^p n)$
 - If $p < 0$, $T(n) = \Theta(n^k)$



Solve the recurrence

$$T(n) = 16T(n/4) + n^3$$

CASE 1: $\text{Log}_b a > k$

$$T(n) = \Theta(n^{\log_b a})$$

CASE 2: $\text{Log}_b a = k$

- If $p > -1$, $T(n) = \Theta(n^k \log^{p+1} n)$
- If $p = -1$, $T(n) = \Theta(n^k \log(\log n))$
- If $p < -1$, $T(n) = \Theta(n^k)$

CASE 3: $\text{Log}_b a < k$

- If $p \geq 0$, $T(n) = \Theta(n^k \log^p n)$
- If $p < 0$, $T(n) = \Theta(n^k)$

1. $a=16, b=4, f(n) = n^3$

2. Find the values of k and p

3. $f(n) = \Theta(n^k \log^p n)$

4. In the problem $f(n) = n^3$

Compute and find k and p

5. $k=3, p=0$

6. Find $\text{Log}_b a = \text{Log}_4 16 = 2$

7. Compare $\text{Log}_b a$ with k

Here it ($2 < 3$) falls on case 3

8. Check with p value it is ≥ 0

9. $\Theta(n^3 \log^0 n)$

10. $\Theta(n^3)$

$$T(n) = 4T(n/2) + n^2 \log n$$

- Try yourself....





$$T(n) = T(n-1) + n \quad \text{----} \quad \Theta(n^2)$$

- Recursive algorithm that loops through the input to eliminate one item- Bubble sort using Recursion

Algorithm Bubble sort(A,N)	
{	
if (n==1) return;	1
// One Pass of Bubble sort, Largest element placed in end	
for (int i=0; i<n-1; i++)	N
{	
if (arr[i] > arr[i+1])	N-1
swap(arr[i], arr[i+1]);	
}	
// Recursion for remaining array	
Bubble sort(A, N-1)	
}	



$$T(n) = T(n-1) + n$$

$$= T(n-2) + n-1 + n$$

$$= T(n-3) + n-2 + n-1 + n$$

// Generalize to k

$$= T(n-k) + n-k+1 + n-k+2 + \dots + n-1 + n$$

// since $n-k = 1$ so $T(1) = 1$

$$= 1 + 2 + \dots + n$$

$$// \text{Here} = n(n-1)/2 = n^2/2 - n/2$$

// By Taking the dominating term which is $n^2/2$

$$= \mathbf{O(n^2)}$$



Finding Max & Min – General solution

Alg Generalmaxmin(a, n, max, min)

```
{  
    max=min=a[1];  
    for i=2 to n do  
    {  
        if(a[i]>max) then max =a[i];  
        if(a[i]<min) then min=a[i];  
    }  
}
```

Time : $2(n-1)$ comparisons in best, average and worst cases



Finding Max & Min – Improvement

Alg Generalmaxmin(a, n, max, min)

```
{  
    max=min=a[1];  
    for i=2 to n do  
    {  
        if(a[i]>max) then max =a[i];  
        else if(a[i]<min) then min=a[i];  
    }  
}
```

Best case occurs when elements are in increasing order $-(n-1)$ comparisons

Worst case occurs when elements are in decreasing order $-2(n-1)$ comparisons

Average case occurs greater than max half the time- $(3n/2 - 1)$



Divide and Conquer Strategy- Finding Max & Min

```

Algorithm MaxMin(i, j, max, min)
// a[1 : n] is a global array. Parameters i and j are integers,
//  $1 \leq i \leq j \leq n$ . The effect is to set max and min to the
// largest and smallest values in a[i : j], respectively.
{
    if (i = j) then max := min := a[i]; // Small(P)
    else if (i = j - 1) then // Another case of Small(P)
    {
        if (a[i] < a[j]) then
        {
            max := a[j]; min := a[i];
        }
        else
        {
            max := a[i]; min := a[j];
        }
    }
    else
    { // If P is not small, divide P into subproblems.
      // Find where to split the set.
      mid :=  $\lfloor (i + j) / 2 \rfloor$ ;
      // Solve the subproblems.
      MaxMin(i, mid, max, min);
      MaxMin(mid + 1, j, max1, min1);
      // Combine the solutions.
      if (max < max1) then max := max1;
      if (min > min1) then min := min1;
    }
}
    
```

Analysis of Max & Min

A[1]	2	3	4	5	6	7	8	9
20	3	-15	-8	25	80	14	51	27

Problem is big

		80	-15
1	9	Max	Min

Problem is big

		25	-15
1	5	Max	Min

		80	14
6	9	Max	Min

		20	-15
1	3	Max	Min

		25	-8
4	5	Max	Min

		80	14
6	7	Max	Min

		51	27
8	9	Max	Min

		20	3
1	2	Max	Min

		-15	-15
3	3	Max	Min

Problem size : 2 elements

Problem size : 1 element



Analysis of Max & Min

- **What is the number of element comparisons required??- The recurrence relation is**

$$T(n) = \begin{cases} T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + 2 & n > 2 \\ 1 & n = 2 \\ 0 & n = 1 \end{cases}$$

- **By solving this, we get $(3n/2 - 2)$ Comparisons required for Best, Worst and average cases**
- **In case of storage, this is not better because it requires stack space for $(i, j, \max, \min, \max I, \min I)$**

